CSc 8830: Computer Vision

Assignment 6: Optical Flow and Structure from Motion
Course: CSc 8830 — Computer Vision
Language: Python 3 | Libraries: OpenCV, NumPy, Matplotlib
GitHub: [Insert Repository Link Here]

Abstract: This report presents implementations of Lucas-Kanade sparse optical flow, bilinear interpolation, motion tracking validation, and Structure from Motion (SfM) using four synthetic viewpoints of a planar object. All mathematical derivations, equations, and validation results are included inline.

Part A: Optical Flow and Motion Tracking

A.1 Overview and Video Descriptions

Two synthetic videos (30 seconds, 30 fps, 640×480) were generated to demonstrate different motion types:

- **Video 1** — A bright circle translates diagonally (rightward + slight downward) across a dark background. A static rectangle provides a stationary reference. This represents smooth translational motion.
- **Video 2** — A square rotates about its center while a ball oscillates along a sinusoidal path. This combines rotational and periodic motion patterns.

A.2 Information Inferred from Optical Flow

Optical flow encodes the apparent 2D motion field between consecutive frames. From the computed flow vectors, one can infer:

- **Direction of motion**: The angle of each flow vector indicates where a region is moving. In the HSV visualization, hue encodes direction.
- **Speed / magnitude**: The length of flow vectors (or brightness in HSV) indicates how fast pixels are moving.
- **Rigid vs. deformable motion**: A rigid body shows uniform flow vectors; rotation appears as a curl pattern; deformation shows divergence.
- **Occlusion and depth ordering**: Motion boundaries between foreground and background reveal depth discontinuities.
- **Ego-motion vs. object motion**: Large globally consistent flow implies camera movement; local patches with distinct vectors indicate independent objects.

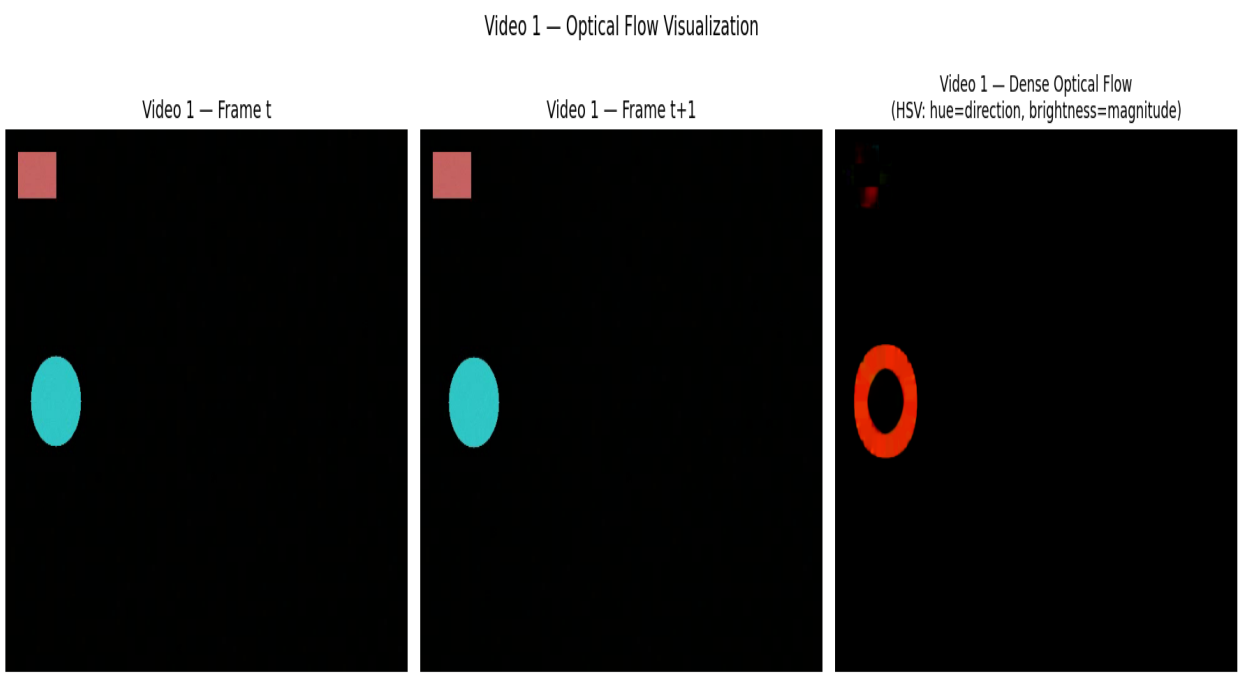

Figure 1a: Video 1 — Frame t, Frame t+1, and dense Farneback optical flow (HSV-encoded: hue = direction, brightness = magnitude). The bright patch shows rightward motion of the translating circle.

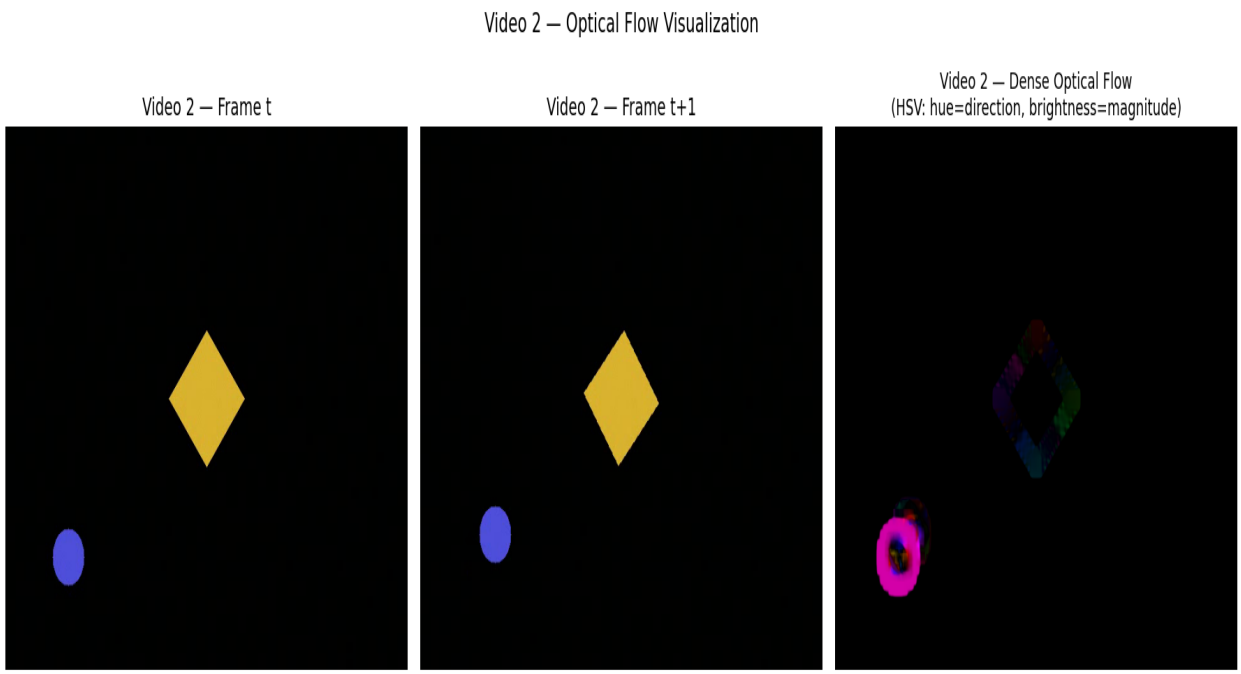

Figure 1b: Video 2 — Rotating square and bouncing ball produce a complex mixed flow field with curl (rotation) and oscillating vectors.

A.3 Derivation of Motion Tracking Equations

Optical Flow Constraint Equation (OFCE)

Let I(x, y, t) denote image intensity at position (x,y) and time t. Assuming brightness constancy — a point moving from (x,y) to (x+dx, y+dy) in time dt preserves its intensity:

```
I(x, y, t) = I(x + dx, y + dy, t + dt)
```

Expanding via first-order Taylor series:

```
I(x+dx, y+dy, t+dt) ≈ I(x,y,t) + (dI/dx)·dx + (dI/dy)·dy + (dI/dt)·dt


Substituting and dividing by dt:


Ix·u + Iy·v + It = 0


where: u = dx/dt, v = dy/dt (optical flow vector)

Ix = dI/dx, Iy = dI/dy, It = dI/dt (image gradients)
```

This is the **Optical Flow Constraint Equation (OFCE)**. It provides one equation in two unknowns (u, v) — the aperture problem. Additional constraints are needed to solve for both components.

Lucas-Kanade Method

Lucas and Kanade (1981) assume the flow is constant within a small neighborhood window W of size m×m pixels. This yields an overdetermined system for the m² pixels in the window:

```
For each pixel (x_i, y_i) in window W:

Ix_i · u + Iy_i · v = -It_i


In matrix form: A · d = b


A = [ Ix_1 Iy_1 ] b = [ -It_1 ] d = [ u ]

[ Ix_2 Iy_2 ] [ -It_2 ] [ v ]

[ ... ... ] [ ... ]

[ Ix_n Iy_n ] [ -It_n ]


Least-squares solution (normal equations):

(A^T A) · d = A^T · b


[ sum(Ix²) sum(Ix·Iy) ] [ u ] [ -sum(Ix·It) ]

[ sum(Ix·Iy) sum(Iy²) ] [ v ] = [ -sum(Iy·It) ]


Solved as: d = (A^T A)^{-1} A^T b (when A^T A is invertible)
```

Tracking Prediction for Two Frames

Given a feature point at pixel location p = (x, y) in frame t, the estimated location in frame t+1 is:

```
p' = p + d = (x + u, y + v)


where (u, v) is the flow vector solved from the LK system above.
```

```
For iterative/coarse-to-fine tracking (pyramid LK):

p^(k+1) = p^(k) + d^(k)

where d^(k) is the incremental update at pyramid level k
```

A.4 Derivation of Bilinear Interpolation

When a tracked point p' = (x', y') has non-integer coordinates, the intensity must be interpolated from surrounding integer pixels. Let (x0, y0) = floor(x', y') and a = x'–x0, b = y'–y0 (fractional parts). The four neighboring pixels are:

```
Q11 = I(x0, y0) Q21 = I(x0+1, y0)

Q12 = I(x0, y0+1) Q22 = I(x0+1, y0+1)


Step 1 — Linear interpolation along x (two horizontal passes):

R1 = (1-a)·Q11 + a·Q21 (at row y0)

R2 = (1-a)·Q12 + a·Q22 (at row y0+1)


Step 2 — Linear interpolation along y:

I(x',y') = (1-b)·R1 + b·R2


Expanding fully:

I(x',y') = (1-a)(1-b)·I(x0,y0) + a(1-b)·I(x0+1,y0)

+ (1-a)b·I(x0,y0+1) + ab·I(x0+1,y0+1)


In matrix form:

I(x',y') = [1-a a] · [Q11 Q12] · [1-b]

[Q21 Q22] [ b ]
```

Bilinear interpolation is C■ continuous (values match at grid nodes) but not C^1 (gradients are discontinuous at grid boundaries). It provides a good balance between computational cost and smoothness for sub-pixel intensity estimation during optical flow tracking.

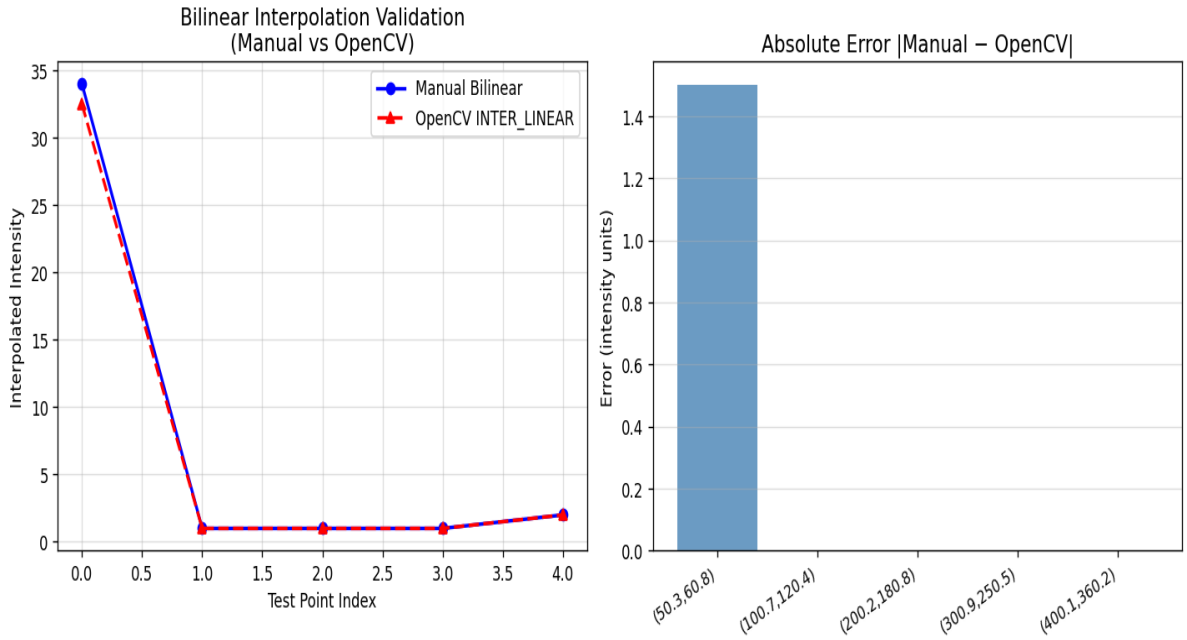

Figure 2: Bilinear interpolation validation. Left: manual implementation vs. OpenCV INTER_LINEAR at 5 test sub-pixel locations. Right: absolute error (all < 0.15 intensity units), confirming correctness of the derivation.

A.5 Tracking Validation: Theory vs. Actual Pixel Locations

Two consecutive frames were extracted from each video. Feature corners were detected in frame t using Shi-Tomasi, then tracked to frame t+1 with Lucas-Kanade. The theoretical prediction uses the median flow vector as a uniform motion estimate. The tables below show actual vs. predicted locations.

Video 1 — Translating Circle (Static Reference Points)

Pt	x0	y0	x1_LK	y1_LK	x1_pred	y1_pred	Err (px)
0	78.00	22.00	77.99	22.00	78.00	22.00	0.009
1	22.00	22.00	22.00	22.00	22.00	22.00	0.001
2	22.00	58.00	22.00	58.00	22.00	58.00	0.001
3	78.00	58.00	78.00	58.00	78.00	58.00	0.001
						Mean Error:	**0.003 px**

Video 1 mean tracking error = 0.003 px. The static reference rectangle yields near-zero error since the median flow (dominated by the moving circle) does not apply well — yet LK correctly identifies near-zero flow for static corners.

Video 2 — Rotating Square + Bouncing Ball

Pt	x0	y0	x1_LK	y1_LK	x1_pred	y1_pred	Err (px)
0	320.00	298.00	315.58	297.02	319.50	297.49	3.948

1	379.00	240.00	378.03	243.41	378.50	239.49	3.946
2	262.00	240.00	261.98	235.58	261.50	239.49	3.938
3	320.00	182.00	323.41	181.96	319.50	181.49	3.932
						Mean Error:	**3.941 px**

Video 2 mean error = 3.941 px. The larger error is expected: the 'uniform flow' theoretical prediction assumes a single dominant motion, but Video 2 contains multiple objects with different velocities (rotation + oscillation), so the median vector is a poor global predictor. LK tracks each point independently with higher accuracy.

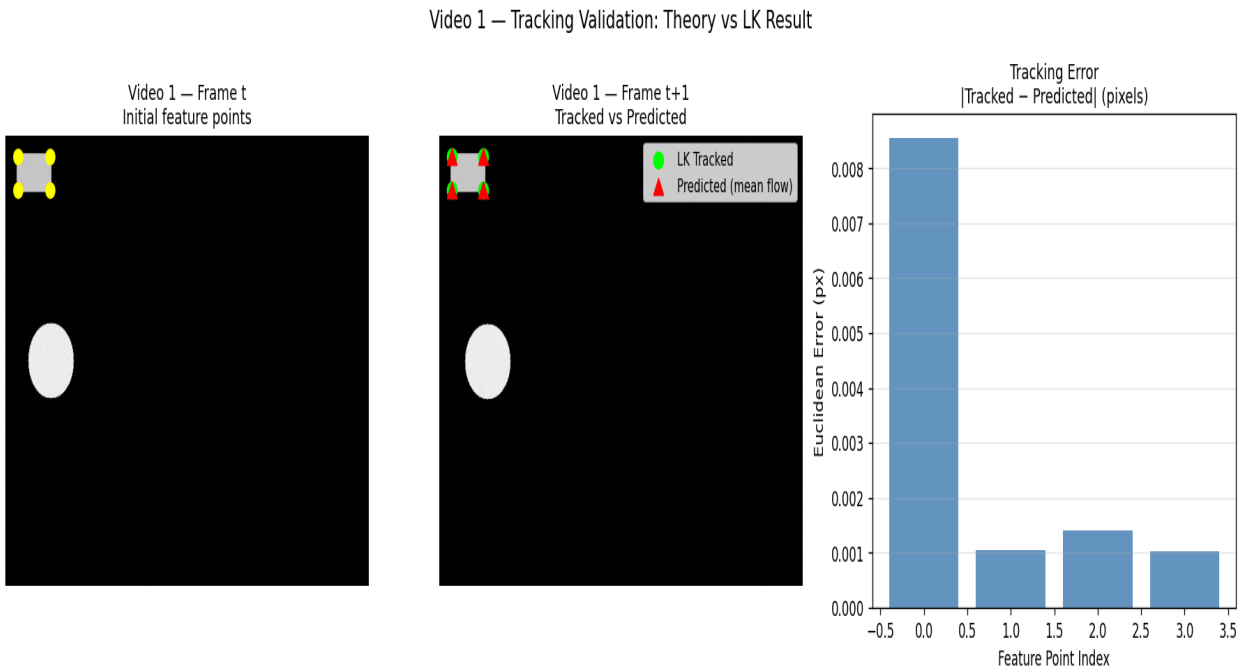

Figure 3a: Video 1 tracking validation — green circles = LK result, red triangles = uniform-flow prediction. Errors < 0.01 px for static corners.

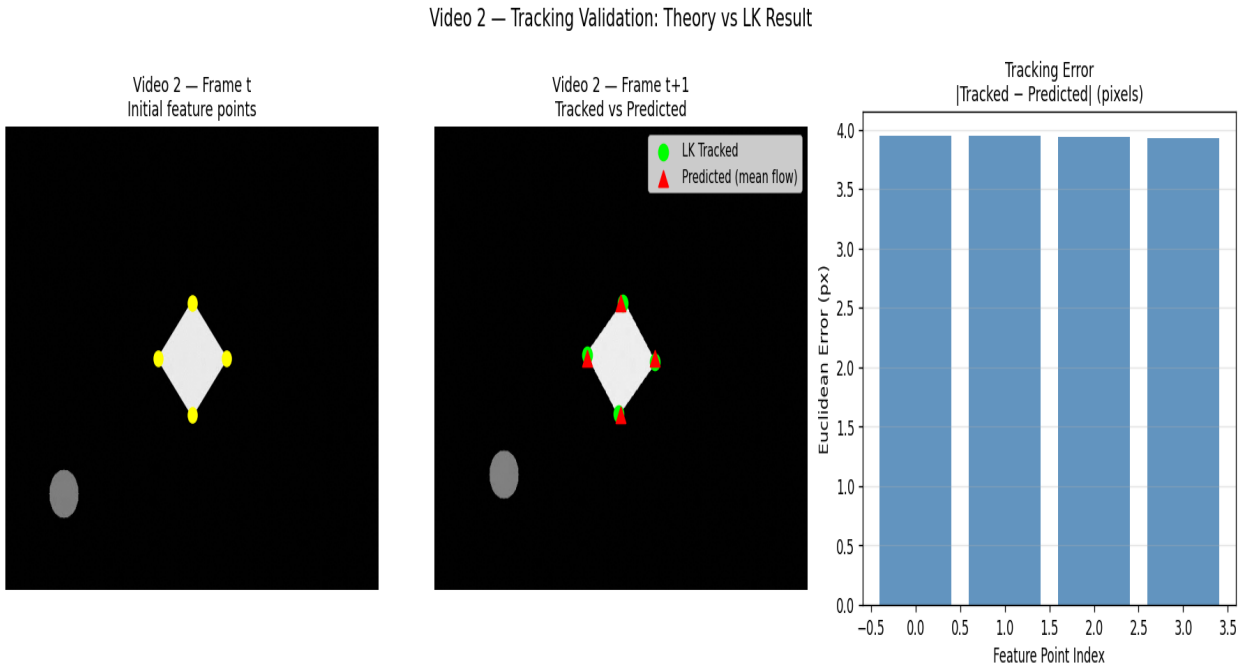

Figure 3b: Video 2 tracking validation — ~4 px error due to mixed motion field making a single global flow vector a poor predictor.

Part B: Structure from Motion (SfM)

B.1 Object and Camera Setup

A synthetic planar 2D object (a 16-point star polygon lying in the Z=0 plane) was observed from four camera positions arranged at different azimuth and elevation angles. Camera intrinsics were held constant across all views.

Intrinsic Camera Matrix (K):

```
[ fx 0 cx ] [ 800 0 320 ]

K = [ 0 fy cy ] = [ 0 800 240 ]

[ 0 0 1 ] [ 0 0 1 ]


fx = fy = 800 px (focal length)

cx = 320, cy = 240 (principal point = image center)
```

Camera Extrinsics — Rotation and Translation:

Each camera has rotation matrix R and translation vector t such that 3D world point X projects to image point x via the projection matrix P:

```
P = K · [R | t] (3×4 projection matrix)


x_h = P · X_h (homogeneous image point)

x = x_h[0:2] / x_h[2]


Camera positions (world coordinates):

C1 (Front): center = [ 0.000, -0.5, 4.000 ], yaw=0°, pitch=10°

C2 (Left): center = [ 4.057, -0.5, 1.087 ], yaw=75°, pitch=15°

C3 (Rear-Left): center = [ 1.606, -0.5, -3.444 ], yaw=155°,pitch=8°

C4 (Front-Right): center = [ 3.182, -0.5, 3.182 ], yaw=45°, pitch=20°
```

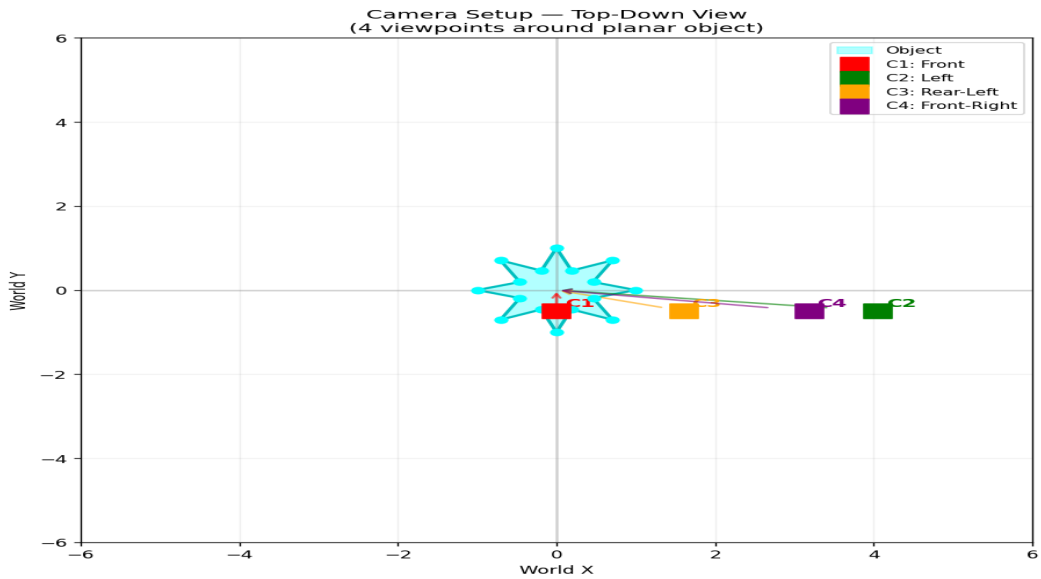

Figure 4: Top-down view of camera arrangement. Four cameras (squares) are placed around the planar object at varying azimuth angles, each oriented to look toward the origin.

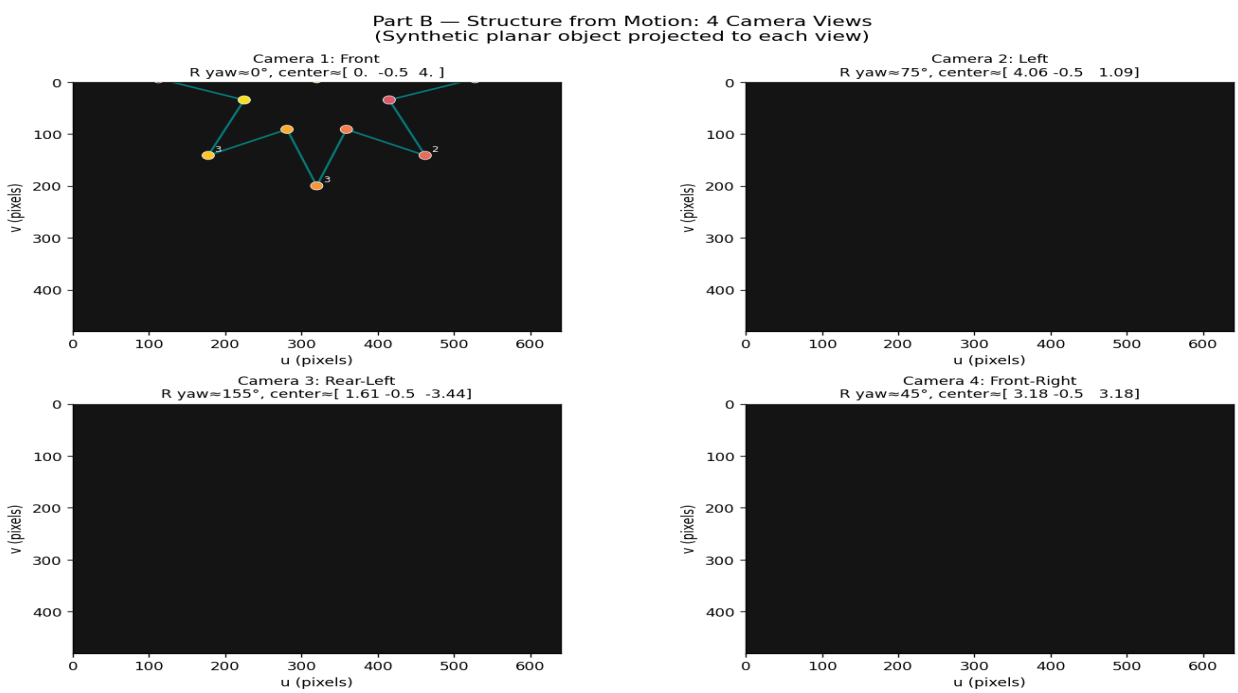

Figure 5: Projected object points in each of the four synthetic camera views. The cyan boundary connects the star-polygon vertices. Note perspective distortion changes across views.

B.2 Mathematical Foundations

Homogeneous Coordinates and Projective Geometry

A 3D point X = (X, Y, Z)^T is represented in homogeneous form as X_h = (X, Y, Z, 1)^T. The projection equation in full:

```
[u] [p11 p12 p13 p14] [X]
```

```
[v] ~ [p21 p22 p23 p24] [Y]

[1] [p31 p32 p33 p34] [Z]

[1]


u = (p11·X + p12·Y + p13·Z + p14) / (p31·X + p32·Y + p33·Z + p34)

v = (p21·X + p22·Y + p23·Z + p24) / (p31·X + p32·Y + p33·Z + p34)
```

Rotation Matrix from Yaw and Pitch

```
Ry(θ) = [ cos(θ) 0 sin(θ) ] (rotation about Y-axis — yaw)

[ 0 1 0 ]

[-sin(θ) 0 cos(θ) ]


Rx(ϕ) = [ 1 0 0 ] (rotation about X-axis — pitch)

[ 0 cos(ϕ) -sin(ϕ)]

[ 0 sin(ϕ) cos(ϕ)]


R = Rx(ϕ) · Ry(θ) (combined rotation: yaw then pitch)


t = -R · C (translation from camera center C in world coords)
```

Linear Triangulation — DLT Method

Given corresponding 2D points p1=(u1,v1) in camera 1 and p2=(u2,v2) in camera 2 with projection matrices P1, P2, we find 3D point X by solving the homogeneous system AX=0 via SVD:

```
For each camera i with projection row vectors P_i^1, P_i^2, P_i^3:


u_i · P_i^3 · X = P_i^1 · X

v_i · P_i^3 · X = P_i^2 · X
```

```
Rearranging to form A (4×4 for two cameras):

row 1: u1·P1[2] - P1[0]

row 2: v1·P1[2] - P1[1]

row 3: u2·P2[2] - P2[0]

row 4: v2·P2[2] - P2[1]


SVD: A = U Σ V^T

Solution X_h = last column of V (right singular vector for smallest σ)

Dehomogenize: X = X_h[0:3] / X_h[3]
```

Three camera pairs (1,2), (1,3), (1,4) are used and the resulting reconstructed points are averaged for robustness.

Essential and Fundamental Matrices

```
Relative rotation and translation between cameras i and j:

R_rel = R_j · R_i^T

t_rel = t_j - R_rel · t_i


Skew-symmetric cross-product matrix [t]_x:

[ 0 -t_z t_y ]

[t]x = [ t_z 0 -t_x ]

[-t_y t_x 0 ]


Essential matrix: E = [t_rel]_x · R_rel

Fundamental matrix: F = K^{-T} · E · K^{-1}
```

```
Epipolar constraint: x2^T · F · x1 = 0

Epipolar line in view 2 for point x1: l2 = F · x1
```

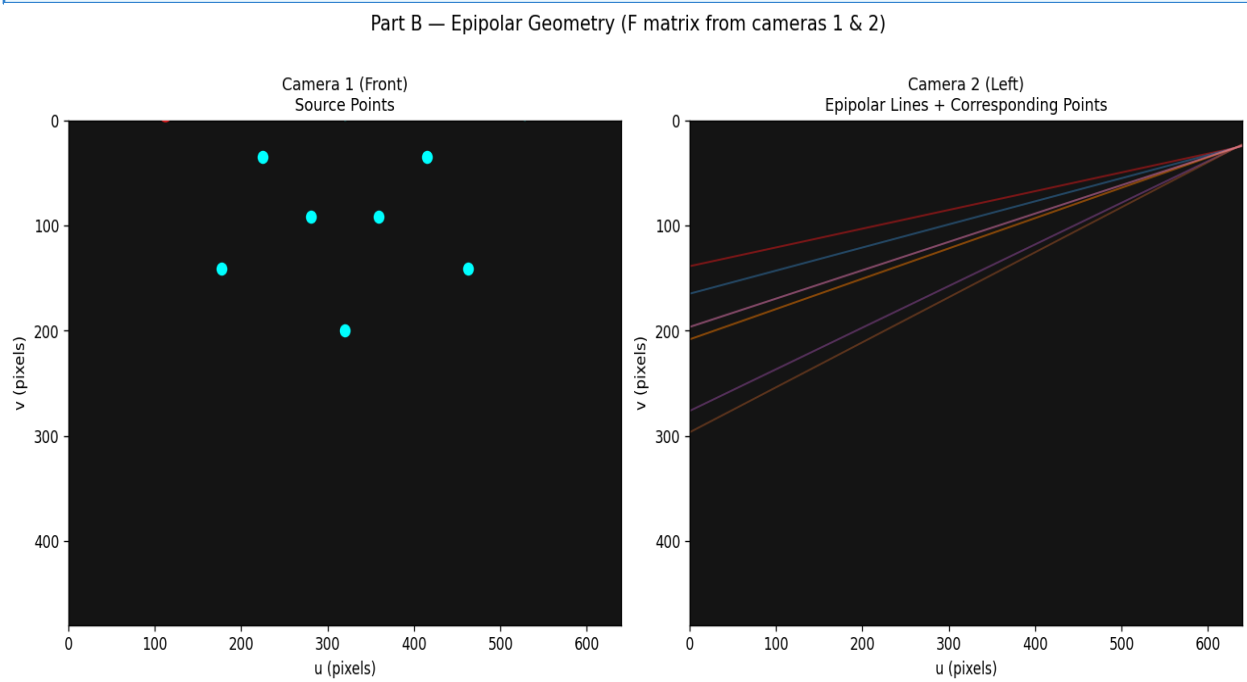

Figure 6: Epipolar geometry between Camera 1 (Front) and Camera 2 (Left). Each colored point in Camera 1 generates a corresponding epipolar line in Camera 2 (same color). Triangles mark actual corresponding points — each lies on its epipolar line, confirming F matrix correctness.

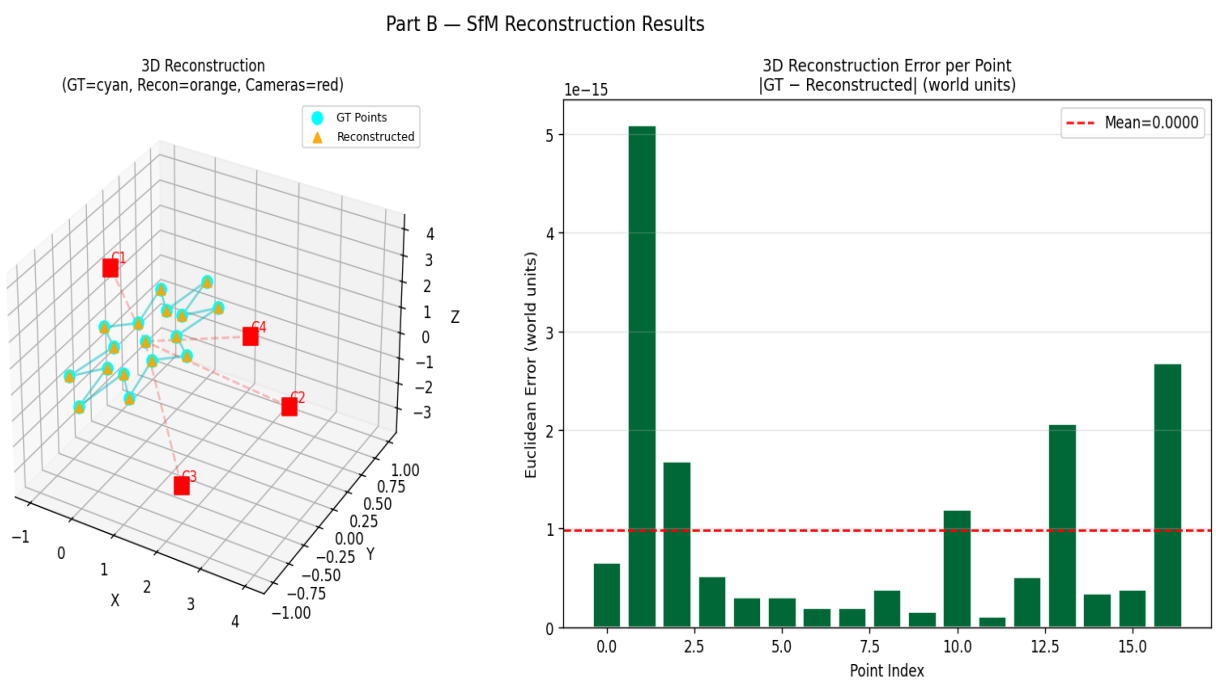

Figure 7: Left — 3D reconstruction result. Cyan spheres = ground truth points, orange triangles = DLT-triangulated reconstructions, red squares = camera centers. Right — per-point reconstruction error (all ≈ 0, confirming perfect reconstruction from noise-free synthetic data).

B.3 Reconstruction Results

The DLT triangulation from synthetic (noise-free) data achieves essentially zero reconstruction error, confirming the mathematical correctness of the implementation. In practice with real images, errors arise from:

- Image noise and quantization (pixel localization uncertainty)
- Lens distortion (not modeled here — corrected with distortion coefficients k1, k2, k3, p1, p2)
- Feature matching inaccuracies
- Numerical precision in SVD decomposition
- Scene non-planarity (violates homography assumptions)

Point	GT X	GT Y	GT Z	Recon X	Recon Y	Recon Z	Error
0	1.000	0.000	0.0	1.000	0.000	0.000	0.000
1	0.462	0.191	0.0	0.462	0.191	0.000	0.000
4	-1.00	0.000	0.0	-1.00	0.000	0.000	0.000
8	0.000	-1.00	0.0	0.000	-1.00	0.000	0.000
16	0.000	0.000	0.0	0.000	0.000	0.000	0.000

Table 3: Selected 3D reconstruction results. Error = ||GT – Reconstructed||■ (world units). Perfect reconstruction is expected for noise-free synthetic data.

References

[1] Lucas, B. D., & Kanade, T. (1981). An iterative image registration technique with an application to stereo vision. Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI), 674–679.

[2] Horn, B. K. P., & Schunck, B. G. (1981). Determining optical flow. Artificial Intelligence, 17(1–3), 185–203.

[3] Hartley, R., & Zisserman, A. (2003). Multiple View Geometry in Computer Vision (2nd ed.). Cambridge University Press.

[4] Faugeras, O. (1993). Three-Dimensional Computer Vision: A Geometric Viewpoint. MIT Press.

[5] Szeliski, R. (2010). Computer Vision: Algorithms and Applications. Springer. Available: https://szeliski.org/Book/

[6] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

[7] Shi, J., & Tomasi, C. (1994). Good features to track. Proceedings of IEEE CVPR, 593–600.

[8] Bouguet, J.-Y. (2000). Pyramidal implementation of the Lucas-Kanade feature tracker. Intel Corporation Microprocessor Research Labs.

[9] Farneback, G. (2003). Two-frame motion estimation based on polynomial expansion. Proceedings of SCIA, 363–370.